

Y8960 TECH DEMO

A sound demo built with C and z88dk
Sound I/O, music data, ADPCM, graphics and tests
0.5.0 / 2026-09-13



Enjoying Y8960 TECH DEMO

Listen

Y8960 is an MSX sound expansion cartridge designed by HRA!.

This demo plays in Y8960-enabled openMSX, which emulates an MSX computer on your PC.

Use keys 1–4 to compare the sections of the approximately 32-second piece.

Watch

White lettering and shadows appear on a blue background, using the MSX 8x8 font by 1re1.

Eight bars indicate activity in the musical parts. They are not measured audio levels.

Starting the demo

Windows

setup-y8960.bat extracts the verified emulator ZIP and runs a startup test.

launch-y8960-tech-demo.bat runs the ROM with isolated settings.

Playing does not require z88dk or Python.

Emulated MSX

C-BIOS MSX2+, Z80, 60 Hz.
A normal V9958 draws the screen.

Cartridge A: 16 KiB demo ROM.
Cartridge B: HRA_Y8960.

No V9968 or proprietary BIOS is used.

What is Y8960?

Adding sound capabilities to MSX

MSX is a computer standard introduced in the 1980s. Expansion cartridges add capabilities.

Y8960 combines several sound generators. The MSX sends commands specifying pitch, timbre and volume.

The environment used here

This demo uses Y8960-enabled openMSX and C-BIOS.

C-BIOS is a free alternative BIOS that provides basic MSX functionality.

No physical MSX or its BIOS ROM is required to start this demo.

How the sound generators work

SSG: simple waveforms

Square waves and noise give electronic sounds a clear outline.

FM: changing waveforms

Operators combine to create metallic or instrument-like timbres. OPLL and OPL2 are FM sound generators.

ADPCM: playing sample data

Sound waveforms are compressed, stored and played back. Here they provide percussion.

Think of the sound generators as musicians, and the program as the instructions for pitch and timing.

PSG and SSG

Feature	PSG	SSG
Name	Programmable Sound Generator	Software-controlled Sound Generator
Typical chip	GI AY-3-8910	Yamaha YM2149
Synthesis	Square waves, noise, volume changes	Same general approach as PSG
Relationship	Name used for standard MSX sound	Yamaha terminology; sound differs
In Y8960	Provides PSG-family functions	Core models Yamaha SSG

SSG and PSG use similar synthesis methods. Sound characteristics vary by implementation.

OPLL and OPL2

Feature	OPLL / YM2413	OPL2 / YM3812
Familiar products	FM-PAC / MSX-MUSIC	Early Sound Blaster / AdLib
Synthesis	Two-operator FM	Two-operator FM
Voices	9 melodic, or 6 + 5 percussion	9 melodic, or 6 + 5 percussion
Timbres	15 presets + 1 custom patch	Program each channel
Custom patches	One setting shared across channels	Separate settings per channel
Typical use	Select a preset and play	Design sounds with parameters

Standard chips only. Y8960 extensions and Sound Blaster PCM playback are separate features.

Why combine sound generators?

Different musical roles

Separate melody, chords, bass and percussion create texture and rhythm.

Layering different synthesis methods produces a sound that one generator alone cannot provide.

The demo adds parts gradually so you can hear the difference.

Example roles

SSG: melodic outline

FM: chords and bass

ADPCM: percussion rhythm

Layer the parts into one piece

Emulator features and demo coverage

Sound roles

SSG: simple electronic waveforms.

OPLL: FM melody and chords.

OPL2: FM bass.

ADPCM: compressed sample playback.

Listen to the same melody with additional layers.

Coverage

This demo uses SSG, OPLL, OPL2 and ADPCM.

It does not use DCSG, SCC or timer-interrupt-driven playback.

It does not exercise every feature or channel of the fork.

Four sections, four layers

01 SEED / 02 MIRROR

SEED: a single SSG melody reveals pitch and articulation.

MIRROR: a second SSG plays the melody four steps behind, adding stereo counterpoint and SSG bass.

The echo replays notes, not recorded audio.

03 ALLOY / 04 PULSE

ALLOY: two OPLL units layer the melody and add chords.

PULSE: adds OPL2 bass and ADPCM kick and snare.

Each section lasts 32 steps, about 8 seconds. The 128-step piece loops.

Writing notes as numbers

Melody and harmony

melody: 64 note numbers; 0 is a rest.

bass_notes: 38, 34, 43, 45.

Four 16-step phrases use a D minor / B-flat / G minor / A harmonic base.

The 64-step material is used twice to form the 128-step performance.

Checks when editing

Reference frequency for note n:

$$f = 440 \times 2^{((n-69)/12)}$$

Pitch tables cover 0–95. Check derived indices such as n–12 and b+31 too.

Timbre, volume and pan are controlled separately in play() / audio_init().

Music timing independent of drawing

The time counter: JIFFY

MSX system software (BIOS) increments this counter once per display refresh cycle.
Its change tells us how much time elapsed.

It advances about 60 times per second, or
50 times with a 50 Hz display setting.
This demo targets about 60 Hz: 15 ticks
(about 0.25 seconds) advance one music step.

128 steps make a loop of about 32 seconds.

Audio first, then a small display update

Read the change in JIFFY to check whether
the next note is due. Music follows time,
not drawing operations, reducing tempo
drift caused by display work.

Send audio commands, then update one
display item. Skip drawing just before a
note or while the display chip is busy.

Audio still needs CPU time, so delays
remain possible.

Enabling sound I/O without unmapping ROM

Slot assumptions

Cartridge B is primary slot 2 in C-BIOS MSX2+.

BIOS WRTSLT (0x0014) writes 0x03 to slot 2 address 0x7FF6 and 0x9F to 0x7FFF.

The C code does not switch out the slot containing its executing ROM.

Checking activation

0x7FF6 enables both OPLL units.

0x7FFF enables OPL2, DCSG, SSGS and the timer.

This demo does not play DCSG.

Write 0x55 to B0 and read it back to check the I/O path, not all sound functions.

Sending register addresses and data

Ports used (hexadecimal)

SSGS: A0 / A1

OPLL 0: 7C / 7D

OPLL 1: 7A / 7B

OPL2 0: C0 / C1

OPL2 1: C2 / C3

Each pair is address / data.

Sending commands from C to the sound chips

ssg(), fm() and opl() write settings to their respective sound generators.

They send the setting number, then its value. Interrupts are briefly disabled so another routine cannot interrupt this pair of writes.

These emulator tests do not guarantee the required delays on hardware or other CPUs.

SSGS pitch, volume and stereo

Pitch and channels

Period $P \approx 1,789,772.5 / (16f)$.

Write the 12-bit period to two registers.

Unit 0 uses base 0x00; unit 1 uses 0x20.

Volume uses offsets 8–10; pan starts at 0x10–0x12.

Note 0 sets volume to zero.

Stereo placement

The melody uses pan 2; the echo uses 13.

SSG bass uses 4, toward the melody.

Channels set to 8 are unused.

The second generator plays the melody from four steps earlier, creating a call and response.

OPLL melody and chords

Pitch data

The generator calculates f-number and block in advance for table lookup.

$$F \approx f \times 2^{(19-\text{block})} / 49,716$$

Send key off, timbre and volume, pitch, then key on.

Layering

Unit 0: preset 3, melody.

Unit 1: preset 4, one octave lower.

Every four steps, send chord notes to other channels in both units.

Using both units does not mean that both are independently panned left and right.

Designing an OPL2 bass sound

Two-operator parameters

0x20/0x23: multipliers and flags.

0x40/0x43: output levels.

0x60/0x63: attack / decay.

0x80/0x83: sustain / release.

0xC0: connection and feedback.

Settings are in `audio_init()`.

Role in this demo

PULSE plays a bass note every two steps.

OPL2 is not limited to bass. This demo uses unit 0 for bass and unit 1 for percussion playback.

Pitch tables are calculated when generating the music data.

Synthesizing percussion in Python

Kick and snare

Kick: a falling sine wave and brief noise.

Snare: noise and a 185 Hz sine wave.

Exponential decay shapes the tail. Random seed 8960 makes generation reproducible.

Sample sizes

Each sound has 4,096 PCM values at 11,025 Hz: about 0.372 seconds.

Encoding as 4-bit ADPCM gives 2,048 bytes per sound, or 4,096 bytes total.

No recordings from commercial sound libraries or existing songs are included.

ADPCM encoding and shared RAM

Encoding

Use the difference from the predicted decoded value to choose sign and magnitude, quantized to four bits.

Update the predicted value and step size, limiting the step to 127–24,576.

Pack two four-bit values into each byte, high nibble first.

Transfer and verification

Send 4,096 bytes to shared sound RAM: kick first, then snare.

The test reads the memory back and compares every byte with the generated data.

It also checks the level of an isolated percussion recording.

Playing ADPCM through two units

Start and end addresses

Address registers use four-byte units.

Kick: start 0, end 511.

Snare: start 512, end 1,023.

The second sound occupies the latter half of shared RAM, not a separate RAM.

Rate and level

Delta-N = 0x38C5.

$49,716 \times \text{Delta-N} / 65,536$

≈ 11,025 samples per second.

Level 175; control 0xA0 starts playback.

Alternate kick and snare every four steps.

Three-color text in SCREEN 5

Screen and cache

SCREEN 5: 256×212, 16 colors.
Page 0 is displayed.

Background 4: RGB(1,2,3).

Text 15: RGB(7,7,7).

Shadow 1: RGB(0,1,1).

At startup, draw headings and status onto
hidden page 1.

White text with a one-pixel lower-right shadow



The CPU builds text and shadow row by row,
then sends nine rows to video memory using
the Z80 repeated output instruction (OTIR).

During playback, the display chip copies
text images prepared at startup.

Drawing commands and interrupts

Small rendering tasks

Each pass updates one heading, status message or activity bar.

Defer drawing just before the next note or while the VDP is busy.

Audio takes priority; display updates may arrive later.

VDP commands

HMMM copies images within video memory. The CPU continues without waiting for completion.

HMMV fills rectangles such as the progress bar.

Check command completion before starting the next drawing operation.

The display update schedule

Choose one item

1. Copy a changed section heading.
2. Update changed playback status.
3. Cycle through bars 1–8 and progress.

Each bar uses two stacked 16×16 sprites: 16 total, eight on a scanline.

Preload height patterns; write two pattern-number bytes to update a bar.

Eight activity bars, enlarged from the demo



Each bar stacks two 16 x 16 pixel sprites. There are 16 total, but only eight across.

A full update takes at least nine passes: about 150 ms at 60 Hz. Updates wait near a note or while the display chip is busy. Each bar decays when it is updated.

ROM, CPU RAM, VRAM and sound RAM

ROM and CPU RAM

ROM: 16 KiB, Normal4000.

BSS ends before 0xC100.

0xC100–0xC107: test state.

0xC200–0xC27F: 128-byte drawing buffer.

No heap is used.

VRAM and sound RAM

Display: page 0, Y=0–211.

Sprites: colors 0x7400, attributes 0x7600,
height patterns 0x7800–0x7A1F.

Headings: page 1, Y=256–383.

Status: nine rows at Y=384, 400, 416.

Sprites do not overlap cached text.

ADPCM RAM is a separate space.

Controls and stopping playback

1–4 / Space / M

1–4: jump to the section start.

Space: pause time and silence; resume sends the current notes.

M: mute while time keeps advancing.

Read the keyboard matrix and act on the initial press edge.

R / Esc and limitations

R: reset step and time to zero.

Esc: silence used sound generators and display stopped status.

The window stays open. Brief key presses during drawing can be delayed or missed.

Current tests use an emulator.

Setup and release files

Two BAT entry points

`setup-y8960.bat`

Checks and installs the required files.

`launch-y8960-tech-demo.bat`

Starts the demo in the prepared environment.

No development tools are needed to play.

Download from GitHub Releases

Extract `y8960-tech-demo-0.5.0.zip`.

Setup obtains the pinned emulator and checks SHA-256 and startup results.

Corresponding sources are supplied on the same Release. Third-party notices are included.

Rebuilding and checking release files

Development build

Install z88dk and Python 3.

Generate music and graphics data, then build the ROM from C source.

The build checks note ranges, ROM size and memory boundaries.

Test entry point

tests/run-all.ps1 checks startup, controls, audio, graphics, ADPCM and note timing.

The release ZIP is built from an allowlist and its contents are compared.

License review is a separate task.

What the automated tests check

Measured timing and limits

Silence to note: 8.507 ms; limit 10.

Boundary error: 8.063 ms; limit 16.688.

Interval error: 23.002 ms.

Timeline drift: 23.003 ms.

The last two limits are 33.376 ms. These tests allow roughly frame-scale delay; they do not promise exact sync.

Rechecked with the current ROM

PASS: 70 seconds, 280 notes, eight boundaries.

Audio-command / recording onset error: 0.291 ms; limit 5 ms.

Tests also check level drops, controls, text and sprites. Host audio-buffer underruns are outside their scope.

Detecting audio interruptions

Negative controls

Detect a missing note, a 100 ms delay and 50 ms of silence.

Also detect a level drop with weak residual audio and a 25 ms recording-clock offset.

Reject recordings from a different ROM or music dataset before analysis.

Coverage limits

Inspect command timing and recorded audio. This does not detect host audio-buffer underruns or every possible noise artifact.

Hardware, other CPUs and 50 Hz configurations require separate tests.

Sources and acknowledgments

Implementation sources

madscient/openMSX_Y8960

Commit: 78469c4

The repository includes C source, music generation tools and tests.

See the development documentation for references and reproduction steps.

Acknowledgments and terms

HRA!: Y8960 design.

buppu3: initial emulation.

madscient: extended fork.

1re1: MSX 8x8 font.

Third-party material retains its own terms.

Checked 2026-09-13.